流水线机器 1 实现总结

IIIF Provider Census and Selector

Boheng Shang

2026-09-16

1. 模块目标与实现内容

流水线机器 1 已实现为可独立运行的 Python 模块 **IIIF Provider Census and Selector**。其职责是建立 IIIF 数据提供机构目录，并为某一次后续处理选择机构。模块不会下载 Manifest，也不会判定 Manifest 版权；这两项分别属于后续资源发现/获取模块与版权识别模块。

本次实现包括：读取 IIIF Registry；沿 `first/next` 或 `last/prev` 遍历 Registry 分页；解析活动中的机构、发现入口及流标签；按发现入口去重；识别 `Add`、`Create`、`Update`、`Remove`、`Delete` 状态；合并人工补充清单；根据命令行中的机构名称进行人为选择；保存来源记录与运行日志。代码只使用 Python 3.10 标准库，无第三方运行依赖。

2. 输入

主输入是 `config.example.json`，其中定义数据来源、网络参数和输出目录；本次机构选择通过命令行参数 `--institution` 输入。数据来源分为两类：

1. **IIIF Registry 来源**：可使用官方 `https://registry.iiif.io/index.json`，也可使用本地的 `OrderedCollection` 或 `OrderedCollectionPage` JSON。程序读取 `orderedItems` 中的活动，并从 `object` 提取发现入口，从 `actor` 提取机构名称和机构网址。
2. **人工补充来源**：`config/manual_providers.json`。由于 IIIF Registry 是自愿维护的目录，可能没有覆盖所有机构，因此可人工补入 `name`、`institution_url`、`discovery_endpoint`、`collection_focus`、`estimated_size` 和版权状态等字段。未知值必须保留为 `null` 或 `"unknown"`，程序不会猜测。

3. 输出与模块间接口

默认将模块间交换数据输出到 `data/output/`，并将运行日志单独输出到 `data/log/`：

文件	用途
`providers.jsonl`	完整、规范化的机构目录；每行一个独立 JSON 对象
`selected_providers.jsonl`	本次选中的机构子集；作为流水线机器 2 的直接输入
`providers.csv`	供人工浏览和表格检查的扁平视图
Module1_YYYY-MM-DD_HH-MM-SS_log.txt	位于 `data/log/`；以电脑本地时间命名，保存选择项、来源、数量及输出路径

4. 内部存储格式

规范化主存储采用 UTF-8 **JSON Lines（JSONL）**。这种格式可以逐行追加或流式读取，也便于后续模块只处理被选中的记录。每条记录包含：稳定的 `provider_id`、机构名称、机构网址、Change Discovery 入口、入口类型、数据流标签、主题范围、估计规模、版权状态、登记状态、最后检查时间、来源信息和原始来源记录。

`provider_id` 由规范化后的发现入口 URL 计算 SHA-256 摘要并截取生成，因此同一入口在多次运行中保持稳定。`raw_source_record` 原样保存 Registry 活动或人工记录；`source` 保存来源类型、来源 URL、分页 URL、活动类型和观察时间，以支持追溯。`record_status = active` 仅表示该登记记录没有被 Registry 标为移除，并不表示入口已通过 HTTP 可访问性检查；`last_checked_at` 因此仍为 `null`，由流水线机器 2 更新。

5. 使用的 API 与验证结果

实现直接依据 **IIIF Change Discovery API 1.0**。程序使用其中的 `OrderedCollection`、`OrderedCollectionPage`、`orderedItems`、`first`、`last`、`next`、`prev` 及 Activity `object` 等结构。机构目录从 **IIIF Registry API/机器入口**读取。网络访问通过 Python 标准库 `urllib.request` 发出 HTTP GET 请求，声明接受 `application/ld+json` 与 `application/json`，并实现超时和有限重试。本模块尚未调用 IIIF Presentation API，因为 Manifest 获取不属于模块 1。

测试命令为 `python -B -m unittest discover -s tests -v`。10 项测试全部通过，覆盖 Registry 分页、活动更新与移除、人工清单、精确机构选择、`all` 选择、错误名称提示、去重、JSONL/CSV 往返、TXT 日志和端到端输出；12 个 Python 文件同时通过语法检查。2026 年 9 月 16 日的真实联网试运行成功读取当前 Registry，得到并选中 1 条登记记录，即 Bodleian Libraries, University of Oxford。该数量只反映 Registry 当时返回的内容，不能视为全球 IIIF 机构总数。官方依据：https://iiif.io/api/discovery/1.0/；Registry：https://registry.iiif.io/。

6. Python 文件及其功能

模块目录当前共有 12 个 .py 文件；其中 9 个是运行代码，3 个是测试代码。

文件	主要功能
iiif_provider_census/__init__.py	定义包的公开接口与版本号，统一导出主要模型、筛选器和流水线入口。
iiif_provider_census/__main__.py	支持通过 python -m iiif_provider_census 启动命令行程序。
iiif_provider_census/cli.py	解析 run、select 命令及参数，调用相应功能并输出结果或错误。
iiif_provider_census/fetch.py	从 HTTP(S) 或本地文件读取 JSON；负责请求头、超时、有限重试和异常封装。
iiif_provider_census/models.py	定义机构记录与来源信息的数据模型；校验 URL、状态和规模并生成稳定机构标识。
iiif_provider_census/registry.py	解析 IIIF Registry、遍历前后分页、读取人工清单、提取机构字段并按入口去重。
iiif_provider_census/selector.py	定义筛选条件，按机构标识、名称、域名、状态和数量上限选择机构。
iiif_provider_census/store.py	读写 UTF-8 JSONL，并导出 CSV 和按时间命名的 TXT 运行日志。
iiif_provider_census/pipeline.py	总调度器：读取配置，依次执行采集、合并、去重、筛选和文件导出。
tests/test_registry.py	测试 Registry 正向/反向分页、活动更新与移除、真实 actor 结构及人工清单。
tests/test_selector_and_store.py	测试筛选规则、稳定排序，以及 JSONL/CSV 保存和读取。
tests/test_pipeline.py	使用本地样例执行端到端测试，检查四类输出及记录数量。

7. 如何运行流水线机器 1

1. 安装 Python 3.10 或更高版本；本模块没有第三方运行依赖。
2. 在 PowerShell 中进入模块目录：

   ```
   cd "E:\exchange-epfl\Semester_Project\Code_git\Module_1"
   ```

3. 按需要编辑 config.example.json。如需补充 Registry 未收录的机构，在 config/manual_providers.json 的 providers 数组中加入记录。
4. 运行完整流程；指定一个机构时输入其完整名称，输入 all 时选择全部当前已知的活动机构：

   ```
   python -B -m iiif_provider_census run --config config.example.json --institution all
   ```

   单一机构示例：--institution "Bodleian Libraries, University of Oxford"。选择多个机构时重复该参数。
5. 在 data/output/ 查看三个数据文件，在 data/log/ 查看 Module1_YYYY-MM-DD_HH-MM-SS_log.txt。其中 selected_providers.jsonl 是交给流水线机器 2 的输入。
6. 验证代码：

   ```
   python -B -m unittest discover -s tests -v
   ```

如只需在已有目录上重新筛选,可运行:python -B -m iiif_provider_census select --input data/output/providers.jsonl --output data/output/selected_providers.jsonl --criteria config/selection.example.json。